

Implementation of Doubly Linked List in C++

Code:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *prev, *next;
    Node(int x) { data = x; prev = next = NULL; }
};

class DoublyList {
    Node *head;
public:
    DoublyList() { head = NULL; }

    void insertPos(int x, int pos) {
        Node *newNode = new Node(x);
        if (pos == 1) {
            newNode->next = head;
            if (head != NULL)
                head->prev = newNode;
            head = newNode;
            return;
        }

        Node *temp = head;
        for (int i = 1; i < pos - 1 && temp != NULL; i++)
            temp = temp->next;

        if (temp == NULL) {
            cout << "Invalid position\n";
            delete newNode;
            return;
        }

        newNode->next = temp->next;
        newNode->prev = temp;
        if (temp->next != NULL)
            temp->next->prev = newNode;
        temp->next = newNode;
    }
};
```

```

}

void deletePos(int pos) {
    if (head == NULL) {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL) {
        cout << "Invalid position\n";
        return;
    }

    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else
        head = temp->next;

    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    delete temp;
}

void display() {
    if (head == NULL) {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;
    while (temp != NULL) {
        cout << temp->data;
        if (temp->next != NULL)
            cout << " <-> ";
        temp = temp->next;
    }
    cout << endl;
}

```

```

};

int main() {
    DoublyList list;
    int choice, x, pos;

    do {
        cout << "\n1. Insert at Position\n2. Delete at Position\n3. Display\n4.
Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter value: ";
                cin >> x;
                cout << "Enter position: ";
                cin >> pos;
                list.insertPos(x, pos);
                break;
            case 2:
                cout << "Enter position: ";
                cin >> pos;
                list.deletePos(pos);
                break;
            case 3:
                list.display();
                break;
            case 4:
                cout << "Exiting...";
                break;
            default:
                cout << "Invalid choice";
        }
    } while (choice != 4);

    return 0;
}

```

Output:

1. Insert at Position
2. Delete at Position
3. Display
4. Exit

Enter choice: 1

Enter value: 4

Enter position: 1

1. Insert at Position
2. Delete at Position
3. Display
4. Exit

Enter choice: 1

Enter value: 5

Enter position: 2

1. Insert at Position
2. Delete at Position
3. Display
4. Exit

Enter choice: 1

Enter value: 6

Enter position: 3

1. Insert at Position
2. Delete at Position
3. Display
4. Exit

Enter choice: 3

4 <-> 5 <-> 6

1. Insert at Position
2. Delete at Position
3. Display
4. Exit

Enter choice: 2

Enter position: 2

1. Insert at Position
2. Delete at Position
3. Display
4. Exit

Enter choice: 3

4 <-> 6

1. Insert at Position
2. Delete at Position
3. Display
4. Exit

Enter choice: 4

Exiting...